

Feature Selection in Statistical Machine Learning

Garvesh Raskutti

Lecture 14

Recap and outline

- Graphical models and feature selection
- Undirected graphical models
- Learning graphs from data as feature selection

What are graphical models?

- Consider nodes/feature $(X_1, X_2, \dots, X_p)$ (Y could also potentially be a node)
- Combines probability/statistics and graph theory
- Edges/non-edges between nodes reflect conditional dependence/independence relationships
- Graph: $G = (V, E)$ where $V = (X_1, X_2, \dots, X_p)$, E is edge set
- Graphs widely used in algorithms/other contexts
- Visual representation of conditional dependence/independence relationships of multi-variate joint distributions
- Directed edges potentially also encode causal/directional relationships
- Examples: time series, spatial/image data, e.t.c.

Why are they interesting/well-studied?

- Visual representation is highly interpretable
- Naturally amenable to many applications (e.g. temporal relationships, spatial relationships, image/language data, e.t.c.)
- Can represent causal/directional relationship
- Simple way to explain seemingly confusing behavior

Two kinds of graphical models

- Undirected graphical models (edges undirected)
- Directed graphical models (edges directed)
- Undirected graphical models represent association/conditional dependence relationships (multi-variate distributions)
- Directed graphical models represent directional/causal/structural relationships (structural equation models)

Undirected graphical models

Undirected graphical models:

$$G = (V, E)$$

where $V = (X_1, X_2, \dots, X_p)$, E undirected edges

Nice and simple relationship:

$$(X_j, X_k) \in E \Leftrightarrow X_j \not\perp\!\!\!\perp X_k | X_S$$

where $S = \{1, 2, \dots, p\} / \{j, k\}$

In words: There exists an edge $(X_j, X_k) \in E$ if and only if you condition on all other nodes/random variables, X_j and X_k are dependent

Problem/goal of interest: Learning graphical model from data

Given observed feature data:

$$\{X_j^{(i)}\}_{j=1,2,\dots,p}^{i=1,2,\dots,n}$$

Problem: Can we learn/estimate the undirected graphical model from observed data?

$$\mathcal{A} : \{X_j^{(i)}\}_{j=1,2,\dots,p}^{i=1,2,\dots,n} \Rightarrow \widehat{G} = (V, \widehat{E})$$

where $V = (X_1, X_2, \dots, X_p)$

Housing data example

$p = 3, n = 300$ features: square footage, number of bedrooms, and a neighborhood quality score (remove price to make it unsupervised). X_1 : Number of bedrooms; X_2 : Square footage; X_3 : Neighborhood quality score. Below is a snapshot of the data.

sqft	bedrooms	neighborhood
1439.326340	3	0.4967141530112327
1370.442884	2	-0.1382643
1952.421519	4	0.6476885381006925
2207.432188	4	1.5230298564080254
2216.778856	5	-0.2341534
1360.536728	3	-0.234137
1532.205659	3	1.5792128155073915
2068.60752	4	0.7674347291529088

Undirected graphical models and conditional independence

For undirected graphical models:

$$(X_j, X_k) \in E \Leftrightarrow X_j \not\perp\!\!\!\perp X_k | X_S$$

or alternatively

$$(X_j, X_k) \notin E \Leftrightarrow X_j \perp\!\!\!\perp X_k | X_S$$

where $S = \{1, 2, \dots, p\} / \{j, k\}$

Shows how undirected graphical models connect to conditional independence but how does it connect to feature selection?

Undirected graphical models and feature selection

For feature selection:

$$X_j \in \tilde{S} \Leftrightarrow Y \not\perp\!\!\!\perp X_j | X_{-j}$$

For undirected graphical models:

$$(X_k, X_\ell) \in E \Leftrightarrow X_k \not\perp\!\!\!\perp X_\ell | X_S$$

where $S = \{1, 2, \dots, p\} / \{k, \ell\}$.

Question: Is there a connection between whether there is a feature X_j in $\tilde{S}$ or there is an edge between $(X_k, X_\ell) \in E$?

Neighborhood selection as feature selection

Define a *neighborhood set* for node X_k :

$$\mathcal{N}(X_k) = \{\ell : \ell \subset \{1, 2, \dots, p\}/\{k\}, (X_k, X_\ell) \in E\}$$

Recall that:

$$(X_k, X_\ell) \in E \Leftrightarrow X_k \not\perp\!\!\!\perp X_\ell | X_S$$

where $S = \{1, 2, \dots, p\}/\{k, \ell\}$. Therefore:

$$\mathcal{N}(X_k) = \{\ell : \ell \subset \{1, 2, \dots, p\}/\{k\}, X_k \not\perp\!\!\!\perp X_\ell | X_S \text{ where } S = \{1, 2, \dots, p\}/\{k, \ell\}\}$$

For feature selection recall that:

$$\tilde{S} = \{j : j \subset \{1, 2, \dots, p\}, Y \not\perp\!\!\!\perp X_j | X_{-j}\}$$

Neighborhood selection as feature selection cont.

For neighborhood selection of X_k in undirected graphical model:

$$\mathcal{N}(X_k) = \{\ell : \ell \in \{1, 2, \dots, p\} \setminus \{k\}, X_k \not\perp\!\!\!\perp X_\ell | X_S \text{ where } S = \{1, 2, \dots, p\} \setminus \{k, \ell\}\}$$

For feature selection of Y recall that:

$$\tilde{S} = \{j : j \in \{1, 2, \dots, p\}, Y \not\perp\!\!\!\perp X_j | X_{-j}\}$$

Therefore:

Neighborhood selection for X_k is equivalent to feature selection where X_k represents Y and the remaining features $\{X_\ell : \ell \in \{1, 2, \dots, p\} \setminus \{k\}\}$ is feature set

And, to learn the full graph structure, run neighborhood selection for all p nodes $(X_1, X_2, \dots, X_p)$.

Summarizing neighborhood selection for undirected graphical model learning

- For each node X_k , $k \in \{1, 2, \dots, p\}$, neighbors (i.e. connected edges) can be learned by performing feature selection where the feature set is all nodes excluding X_k
- Need to learn the neighbors for all p nodes to create entire graph
- Equivalent to solving p feature selection problems

Neighborhood selection code

```
import numpy as np
from sklearn.linear_model import LassoCV
from sklearn.preprocessing import StandardScaler

def neighborhood_selection(X, alpha=0.5):
    n_samples, n_features = X.shape
    adjacency_matrix = np.zeros((n_features, n_features))

    # Standardize data
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    for j in range(n_features):
        # Response is X_j
        y = X_scaled[:, j]
        # Predictors are all other variables
        X_rest = np.delete(X_scaled, j, axis=1)

        # Fit Lasso with cross-validation if alpha is None
        if alpha is None:
            lasso = LassoCV(cv=5, positive=False).fit(X_rest, y)
            coef = lasso.coef_
        else:
            lasso = Lasso(alpha=alpha).fit(X_rest, y)
            coef = lasso.coef_

        # Fill adjacency matrix
        idx = 0
        for k in range(n_features):
            if k == j:
                continue
            if coef[idx] != 0:
                adjacency_matrix[j, k] = 1
            idx += 1

    # Symmetrize adjacency (OR rule)
    adjacency_matrix = np.maximum(adjacency_matrix, adjacency_matrix.T)
    return adjacency_matrix
```

Housing data example

For housing data recall features: X_1 : square footage; X_2 : Number of bedrooms; X_3 : Neighborhood quality score

```
df = pd.read_csv("Lecture_1_HousePrices.csv")
X = df.drop("price", axis=1)
adj = neighborhood_selection(X)
print("Estimated adjacency matrix:")
print(adj)
```

Estimated adjacency matrix:

```
[[0. 1. 0.]
 [1. 0. 0.]
 [0. 0. 0.]]
```

After performing neighborhood selection, the graph adjacency matrix

$$X_1 \not\perp X_2|X_3, X_1 \perp X_3|X_2, X_2 \perp X_3|X_1$$

Interpreting housing data graphical model

- A single edge between X_1 (number of bedrooms) and X_2 (square footage)
- No other edges
- Represents the conditional independence statements: $X_1 \not\perp X_2|X_3$, $X_1 \perp X_3|X_2$, $X_2 \perp X_3|X_1$
- Square footage and number of bedrooms highly dependent, X_3 neighborhood quality independent of (X_1, X_2)

Stock return data example

- Graphical models also useful for stock return data for different S&P 500 companies
- How does stock price of one influence the other given a set of stocks
- For simplicity we consider 8 companies: Apple , JPMorgan, CVX, Johnson and Johnson, Coca cola, Pepsi, Visa, Proctor and Gamble
- Can potentially help you determine which stocks to invest in depending on the goal (i.e. diversification, short term reward maximization, e.t.c.)

Stock return loading data

```
import numpy as np
import pandas as pd
import yfinance as yf
from sklearn.linear_model import LassoCV
from sklearn.preprocessing import StandardScaler
import itertools
import matplotlib.pyplot as plt
import networkx as nx

# -----
# 1. Download data
# -----
tickers = [
    "AAPL",
    "JPM", "CVX", "JNJ",
    "PG", "KO", "PEP",
    "V"
]
|
data = yf.download(
    tickers,
    start="2022-01-01",
    end="2024-01-01",
    auto_adjust=True
) ["Close"]
```

Stock returns pre-processing

```
|: # Compute daily log-returns
   returns = np.log(data).diff().dropna()

   # Standardize
   scaler = StandardScaler()
   X = scaler.fit_transform(returns)
   X = pd.DataFrame(X, columns=tickers)
```

- Log transform pretty standard for stock return data
- Re-scaling also standard especially when fitting Lasso

Stock return neighborhood selection

```
: n, p = X.shape
adj_matrix = np.zeros((p, p))

for j in range(p):
    y = X.iloc[:, j].values
    X_minus_j = X.drop(columns=X.columns[j]).values

    model = LassoCV(cv=10).fit(X_minus_j, y)
    coefs = model.coef_

    idx = 0
    for k in range(p):
        if k == j:
            continue
        if abs(coefs[idx]) > 1e-5:
            adj_matrix[j, k] = 1
            idx += 1

# Symmetrize (OR rule)
adj_matrix = np.maximum(adj_matrix, adj_matrix.T)

: adj_matrix

: array([[0., 1., 0., 1., 0., 1., 1., 1.],
        [1., 0., 1., 1., 1., 0., 1., 0.],
        [0., 1., 0., 1., 1., 1., 1., 0.],
        [1., 1., 1., 0., 1., 0., 1., 1.],
        [0., 1., 1., 1., 0., 1., 1., 1.],
        [1., 0., 1., 0., 1., 0., 1., 0.],
        [1., 1., 1., 1., 1., 1., 0., 1.],
        [1., 0., 0., 1., 1., 0., 1., 0.]])
```

Stock return visualizing graph code

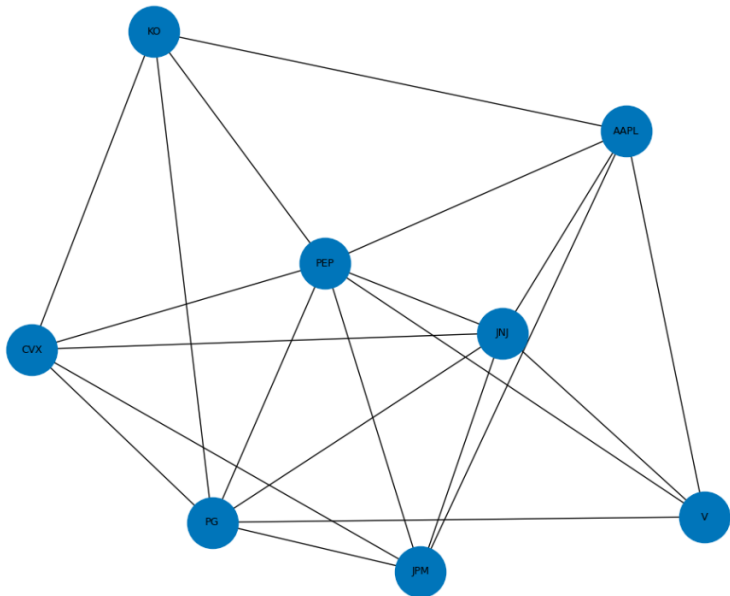
```
G = nx.Graph()

for i in range(p):
    G.add_node(tickers[i])

for i, j in itertools.combinations(range(p), 2):
    if adj_matrix[i, j] == 1:
        G.add_edge(tickers[i], tickers[j])

plt.figure(figsize=(10,8))
pos = nx.spring_layout(G, seed=42)
nx.draw(G, pos, with_labels=True, node_size=2000, font_size=9)
plt.title("Stock Return Graph via Neighborhood Selection")
plt.show()
```

Stock return graph



Interpreting stock return graph and open questions

- **Interpretations:**

- ▶ Pepsi a pretty central node that connects to all others
- ▶ Since Coca Cola and Pepsi are so similar, most returns for Coca Cola are likely mediated through Pepsi

- **Open questions:**

- ▶ How stable is this graph (recall stability for feature selection)?
- ▶ Since we are choosing λ via cross-validation which may over-select, what happens if make λ larger to yield a sparser graph?